Pandas and NumPy

Work with data efficiently • Perform numerical computations • Analyze and manipulate datasets

NumPy | pandas

From arrays to insights!



1. NumPy Basics

- Fast element wise array functions
- Multidimensional Array
- Data Processing using arrays
- File I/O with arrays

1.1 Fast Element wise Array Functions

Element wise operation

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
squared = np.square(arr)
print(squared)  # [ 1  4  9 16 25]
```

Output:

```
[ 1  4  9 16 25]
```

NumPy applies functions to each element of the array at once.

1.2 Multidimensional Array

2D array (rows, columns)

```
import numpy as np
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr.shape)  # (2, 3)
print(arr)
```

Output:

```
(2, 3)
[[1 2 3]
 [4 5 6]]
```

1	2	3
4	5	6

rows

columns

1.3 Data Processing using Arrays

Statistical operations

```
import numpy as np
arr = np.array([10, 20, 30, 40])
mean = np.mean(arr)
std = np.std(arr)
print(mean, std)      # 25.0 11.18
```

Output:

```
25.0 11.180339887498949
```

Use vectorized operations for faster processing.

1.4 File I/O with Arrays

Save and load data

```
import numpy as np
# Saving array to a file
np.savetxt('data.csv', arr, delimiter=',')
# Loading array from a file
loaded_arr = np.loadtxt('data.csv', delimiter=',')
print(loaded_arr)
```

data.csv → save/load → NumPy Array

2. Pandas

- Data Structures
- Essential Functionality
- Summarizing and Computing Descriptive Statistics
- Handling Missing Data
- Hierarchical Indexing

pandas

Organize . analyze . explore data!



2.1 Data Structures

Series & DataFrame

```
import pandas as pd
# Series (1D)
s = pd.Series([10, 20, 30], index=['a','b','c'])
# DataFrame (2D)
df = pd.DataFrame({
    'Name': ['Alice','Bob','Charlie'],
    'Age': [25, 30, 35]
})
print(s)
print(df)
```

Series:

```
a    10
b    20
c    30
dtype: int64
```

DataFrame:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

2.2 Essential Functionality

Load and explore data

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head())    # first 5 rows
print(df.info())    # summary
```

Output (head):

	Name	Age	City
0	Alice	25	Delhi
1	Bob	30	Mumbai
2	Charlie	22	Bangalore

Output (info):

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3):
 #   Column  Non-Null Count  Dtype
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: object(2), int64(1)
```

2.3 Summarizing and Computing Descriptive Statistics

Summary statistics

```
import pandas as pd
df = pd.DataFrame({'Age': [25, 30, 35, 40]})
print(df.describe())
```

Output:

	Age
count	4.000000
mean	32.500000
std	7.905694
min	25.000000
25%	28.750000
50%	32.500000
75%	36.250000
max	40.000000

Provides mean, std, min, max and more.

2.4 Handling Missing Data

Detect and handle missing values

```
import pandas as pd
df = pd.DataFrame({'Name': ['Alice','Bob','Charlie'],
                   'Score': [85, None, 92]})
print(df.isnull())   # Check missing values
df = df.fillna(0)    # Fill missing values
print(df)
```

Output:

	Name	Score
0	False	False
1	False	True
2	False	False

Use fillna(), dropna(), or interpolate().

2.5 Hierarchical Indexing

Multi-level index

```
import pandas as pd
arrays =[['A','A','B','B'],['1','2','1','2']]
index = pd.MultiIndex.from_arrays(arrays, names='Group','Number'))
df = pd.DataFrame({'Values': [10, 20, 30, 40]}, index=index)
print(df)
```

Output:

Group	Number	Values
A	1	10
A	2	20
B	1	30
B	2	40

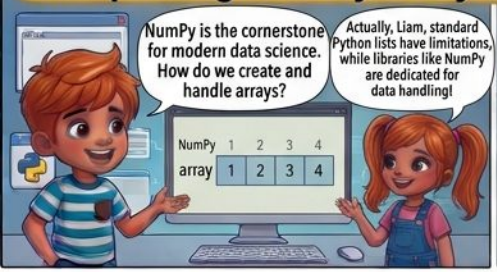
Useful for complex data organization.

Arrays → Clean → Analyze → Visualize → Make Better Decisions

Better data = Better insights!

MASTERING PYTHON DATA SCIENCE: NUMPY BASICS

1. Exploring NumPy Arrays & Data Concepts



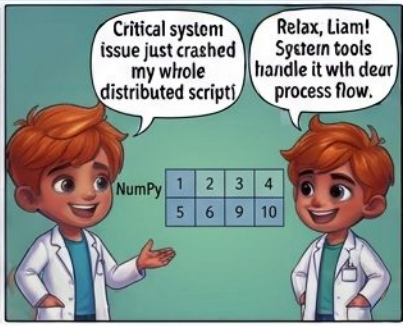
Array Basics & Key Concepts

- Array Creation (np.array)
- Data Types (dtype)
- Indexing & Slicing
- Vectorization

Key terms for content, while cls standardd is powerful, libraries lika Pandas are dedicated data handling tools.

- 'np.array' is a vector. Standard logic is core.
- Slicing is for cleaning. Vectorization is the core.

```
# Data a minimel list
i=teona = [list()
if = rrede.rax(['sbees''])
> print('list)st)
```



2. Interactive Workflow & NumPy Operations



NumPy Array Flow & Operations

Data Access & Connection Flow

Input Data
↓
Array (e.g., np.array)
↓
Transform (e.g., reshaping)
↓
API Operations (e.g., universal functions)
↓
Result

Common built-in operators

shape()	len(), dir() len(), dir()
reshape()	Tix operators: get(), split()
Process	Library operators: get(), split()
num()	Library operators: sum(), mean(()

```
import numpy as np;
arr = np.array([1, 2, 3, 4));
reshaped = arr.reshape(2, 2);
print(reshaped)
```



3. Advanced Array Topics & Workflows



Advanced Array processing & performance

Cell Type	Description
Broadcasting	Boolean Masking
Boolean Masking	Multi-dimensional arrays
Ufuncs & Performance	Memory optimization
Memory optimization	Structured arrays

Workflow Management (Array data flow analysis)

Array DAG



Array Integration Tools

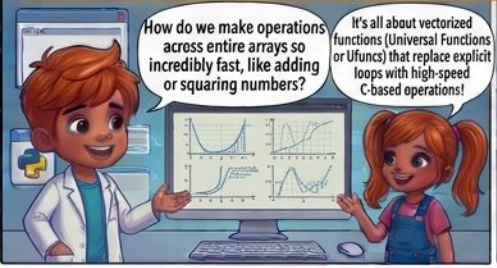
Tool	Usage
numpy	Standard basics
requests	API access from GUI
beautifulsoup4	Web content extraction
json	Token object handling

So, frameworks provide standard primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and scalability. Debugging is key!

Flexible arrays + efficient handling = powerful and reliable data science!

FAST ELEMENT-WISE ARRAY FUNCTIONS & UFUNCS

1. Exploring NumPy Fast Functions & Concepts

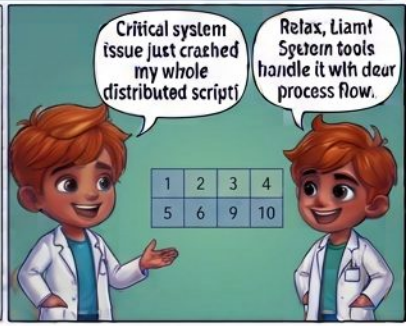


Universal Functions (Ufuncs) & Concepts

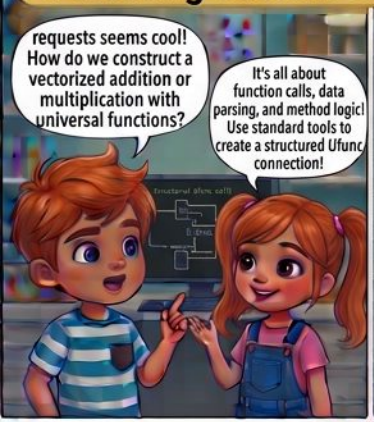
- Fast C-based element-wise functions.
- Work on entire arrays.
- Automatic broadcasting.
- Highly efficient memory use.

- *element-wise* is the concept.
- C implementation is fast.
- broadcasting handles shapes.
- efficiency is core.

```
# Element-wise addition
a = np.array([1, 5])
    b = np.array([7, 4])
result = np.add(a, b)
```



2. Creating Custom Fast Functions & Ufunc Operations



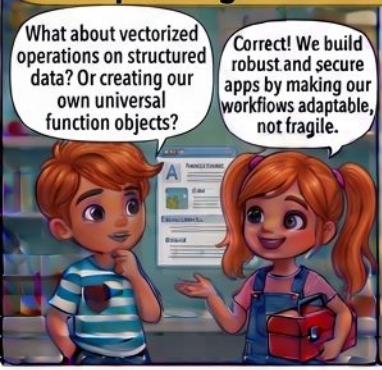
Common Fast Array Libraries & Tasks

Array functions	Library	Description
Array Input	requests	Raw network connection
Array Input	beautifulsoup4	Web content extraction
Reshaping	beautifulsoup4	Data encoding/decoding
Ufunc Operation (e.g., vectorized)	json	Data encoding/ decoding
Result		

```
import request as np;
response = requests.get('api/data');
data = response.json();
print(data)
```



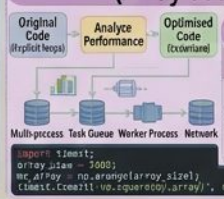
3. Unpacking Advanced Ufunc Topics & Performance



Advanced Ufunc Features & Performance

Cell Type	Description
Vectorization	(replaces loops)
Parallel execution	(multiple cores)
Automatic Broadcasting	
Performance measurement (`%timeit`)	

Workflow & Performance (Array data flow analysis)



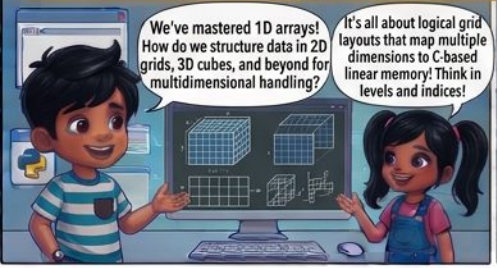
Array Integration Tools	
requests	API access from GUI
sqlalchemy	Powerful ORM/DB connecting
beautifulsoup4	Web content extraction
numpy	Fast element-wise basics

So, frameworks provide core primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and performance optimization. Monitoring is key!

Scalable arrays + efficient handling = powerful and reliable data science!

MULTIDIMENSIONAL ARRAY BASICS

1. Understanding Multidimensional Array Concepts



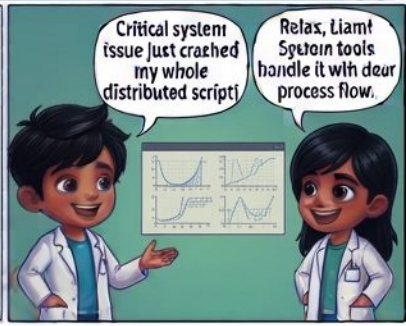
Multidimensional Structure & Indexing

- Structured data grids.
- Level-wise organization.
- Automatic dimension handling.
- High logical complexity.

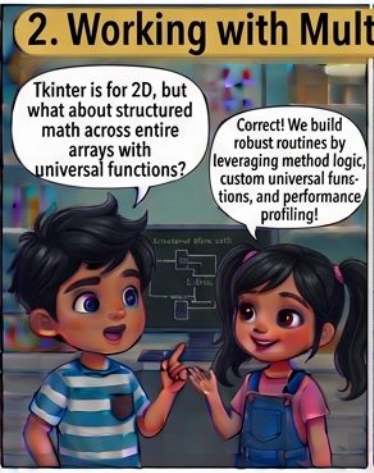
- *Multidimensional* is the concept.
- Indices are levels.
- C implementation is core.
- logic complexity is key.

```
# 3D and 2D arrays
arr2d = np.array([[1, 2],

arr3d = np.array([[1, 2],
                  [3, 5], [7, 8]])
```



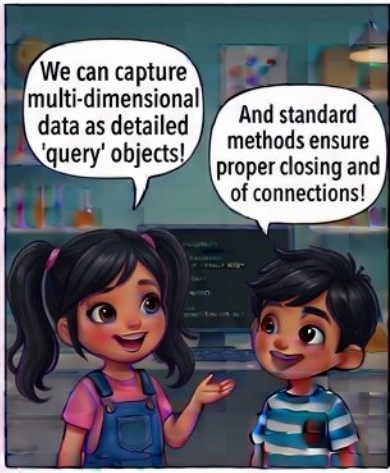
2. Working with Multidimensional Operations



Common Built-in operators

Array functions	Library	Description
Multidimensional	len()	dir()
Input	np ops:	np.add(), np.subtract()
Reshaping	np advanced:	np.dot(), np.tensordot()
Operations (e.g., matrix logic)	np linear algebra:	np.linalg.inv()
Result		

```
import numpy as np;
arr = np.array([[1, 2], [3, 4]]);
inv = np.linalg.inv(arr);
print(inv)
```



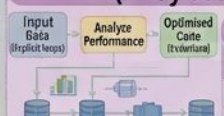
3. Advanced Multidimensional Topics & Performance



Advanced Array Operations & Performance

Cell Type	Description
Slicing & Indexing	(multi-level access)
Broadcasting	(efficient math)
Join & Split	(combining/splitting data)
Reshaping & Flattening	

Workflow & Performance (Array data flow analysis)




```
import numpy as np;
a = np.array([1, 2]);
b = np.vstack(a, 4)); c = np.vstack((a, b)); print(c)
```

Array Integration Tools

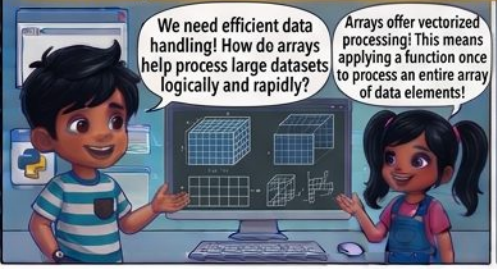
pandas	Powerful handling
requests	API access from GUI
beautifulsoup4	Web content extraction
numpy	Fast element-wise basics

So, frameworks provide standard primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and performance optimization. Monitoring is key!

Flexible arrays + efficient handling = powerful and reliable data science!

DATA PROCESSING & ANALYSIS WITH ARRAYS

1. Array-Based Data Processing Concepts



Core Concepts & Techniques

- **Vectorization** (replaces explicit loops)
- **Broadcasting** (efficient shape handling)
- **Slicing & Indexing** (targeted access)
- **Data Aggregation** (sums, means)

* vectorization* is processing.
- arrays are level-wise.
- memory allocation is efficient.
- logic complexity is core.

```
# Process elements in bulk
array = np.array([1, 2, 3])
result = array * 10
        # Element-wise logic
```



2. Working with Data Operations & Commands



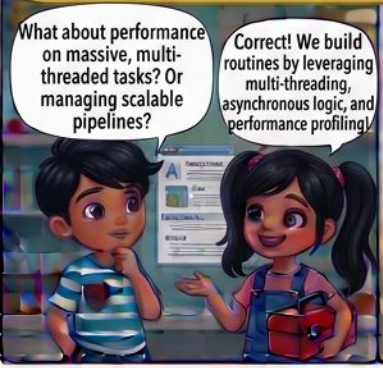
Data Processing Commands

Logic flor data	Processing Command	Usage
Aggregates	np.mean()	Data aggregation
Transforms	requests	external API access
Selection	sqlalchemy	Database integration
Output	psycopg2	External DB driver

```
import numpy as np;
processed_array = np.add(my_data, factor)
result_oean = np.mean(processed_array);
print(result_mean)
```



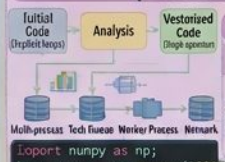
3. Advanced Processing Workflow & Performance



Advanced Processing & Optimization

Coll Type	Description
Parallel execution	(multi-threading)
Memory Optimization	techniques
GPU Acceleration	(for specific tasks)
Workflow standard:	Pipelines

Workflow & Performance (Processing analysis)



Array Integration Tools

numpy	Efficient processing
pandas	Powerful handling
beautifulsoup4	External web content processing
psycopg2	Database driver connection

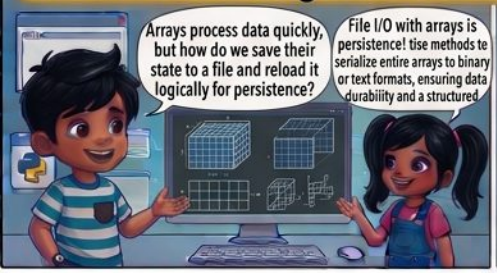
```
import numpy as np;
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```

So, frameworks provide standard processing primitives. Management tools handle data-objects and resources, and a good design with integration tools ensures custom logic and performance. Monitoring is key!

High performance + efficient workflows = powerful and reliable data handling!

FILE I/O WITH ARRAYS – PERSISTENT DATA HANDLING

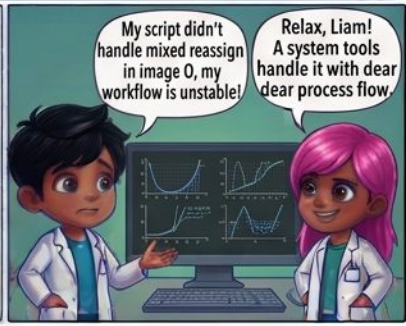
1. Understanding File I/O & Array Persistence



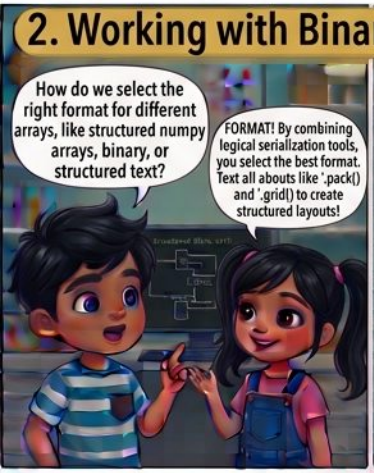
Core Concepts & Techniques

Vectorization (replaces explicit loops)	* vectorization* is processing.
Broadcasting (efficient shape handling)	• arrays are level-wise.
Slicing & Indexing (targeted access)	• memory allocation is efficient.
Data Aggregation (sums, means)	• logic complexity is core.

```
# Process elements in bulk
array = np.array([1, 2, 3])
result = array * 10
         # Element-wise logic
```



2. Working with Binary and Structured File Operations



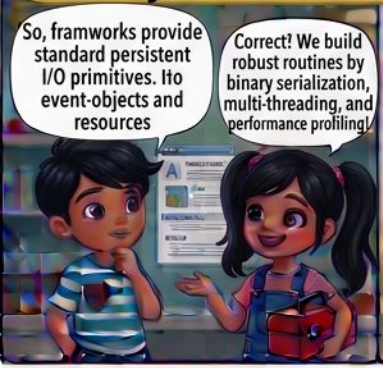
Data Processing Commands

Logic flor data	Processing Command	Usage
Aggregates	np.mean()	Data aggregation
Transforms	requests	external API access
Selection	sqlalchemy	Database integration
Output	psycopg2	External DB driver

```
import numpy as np;
processed_array = np.add(my_data, factor)
result_mean = np.mean(processed_array);
print(result_mean)
```



3. Array Persistence & Workflow Durability

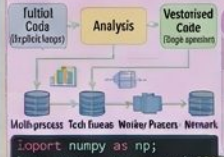


Advanced Ufunc Features & Performance

Cell Type	Description
Parallel execution	(multi-threading)
Memory Optimization	techniques
GPU Acceleration	(for specific tasks)
Workflow standard:	Pipelines

File Workflows & Integrity

(data persistent logical durability analysis)



Array Integration Tools

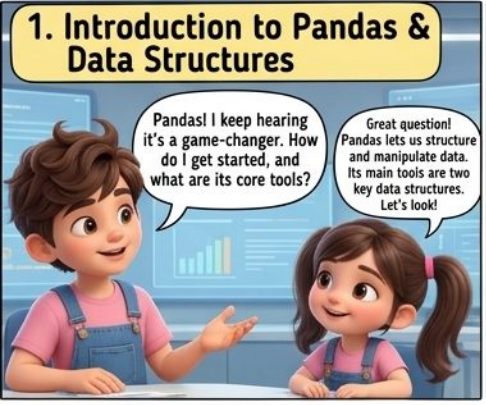
numpy.save	Array serialization
open('w')	Text file writing
np.fromfile	Raw data loading
pickle	Object persistence

```
ioport numpy as np;
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```

So, frameworks provide standard persist: I/O primitives. Management tools handle data-objects and resources, and a good design with integration tools ensures custom logic and performance. Monitoring is key!

Fast File I/O + reliable workflows = dvorkflows = durable persistent!

1. Introduction to Pandas & Data Structures



Pandas Core Structures

Series (1D)

A labeled array.
(like a list)

Example:
Index: 0, 1, 2, 3
Data: A, B, C, D

DataFrame (2D)

A labeled table.
(like a spreadsheet)

Index: 0, 1
Columns: [Name, Age, City]
Data: [[Alice, 25, NY], [Bob, 30, SF]]

Series are like dictionary columns.
A DataFrame is a dictionary of Series.



2. DataFrame Creation: From Dictionaries and Lists



Examples of DataFrame Creation

Production Rules (for creation logic, simplified):
If a dictionary of lists is given → **pd.DataFrame(dict)**.
If a list of dictionaries is given → **pd.DataFrame(list)**.

Creating from Dictionary
(Dictionary approach)

Example Data (Dictionary):
{'A': [1,2], 'B': [3,4]}'

	'A	'B'
0	1,2	3,4
2	3,4	3,4

Good when you have columns ready as lists.

Creating from List
(List approach)

Example Data (List of Dictionaries):
[{'A': 1, 'B': 3}, {'A': 2, 'B': 4}]

	'A	'B'
0	1	3
2	2	4

Good when you have data in rows as records.



3. Dataframe Operations: Data Cleaning vs. Data Analysis



Data Cleaning
(Manipulation)

How much can it *fix*?
- Focus: Correcting data.
- Simpler structures = faster processing.

Examples:
- dropna()
- fillna()
- rename()
- set_index()

Data Analysis
(Computation)

How well does it *calculate*?
- Focus: Deriving insights.
- More complex relationships, less memory.

Examples:
- describe()
- groupby()
- sort_values()
- mean()

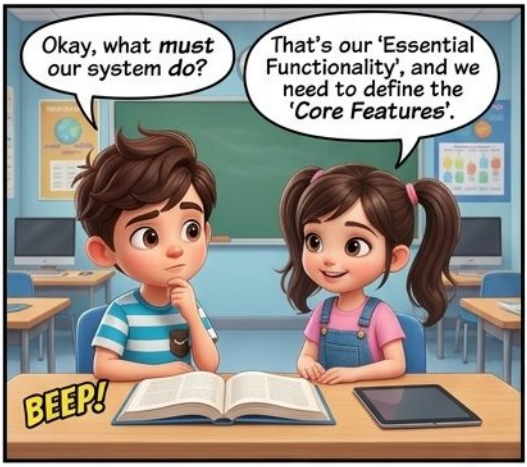
Comparison Table

Common Operations Comparison

Operation Type	Expressiveness (Cleaning complexity)	Efficiency (Computation speed)
dropna()	dropna=manipulation	high cleaning detail
mean()	high	high
groupby()	manipulation	low calculation
fillna()	calculation	low calculation

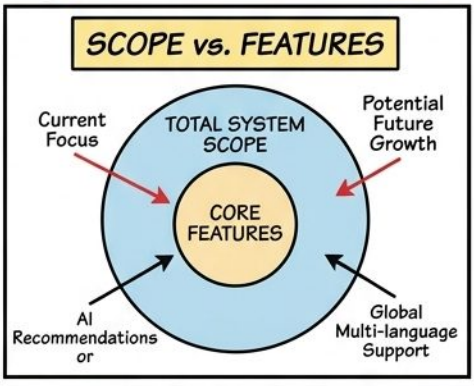
Right knowledge (Problem Type) + Right operations (Pandas function) = Correct Insights! 🙂.

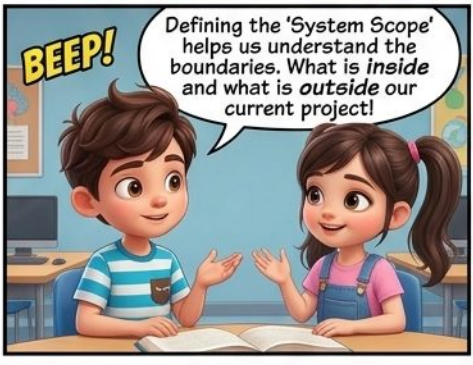
1. Essential Functionality: Core Features & Scope



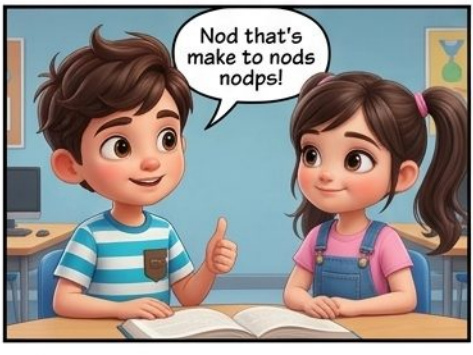


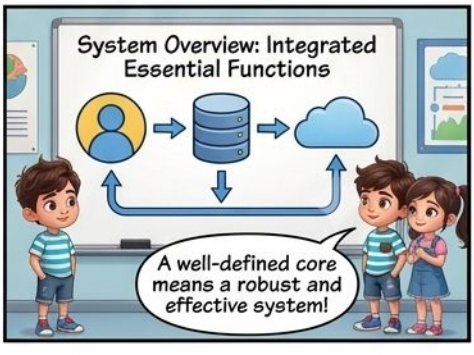






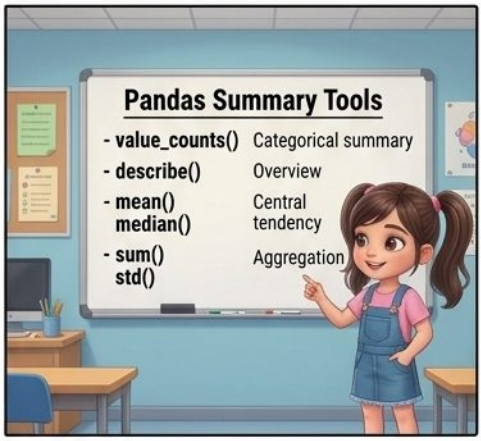




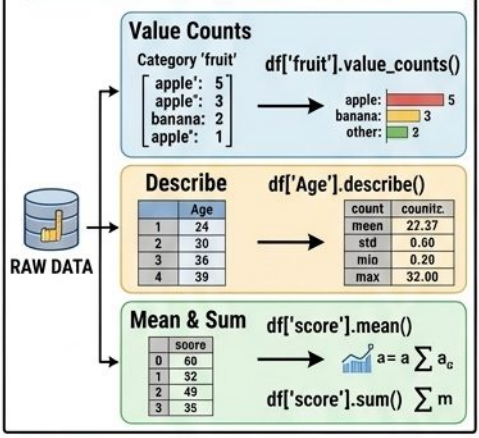


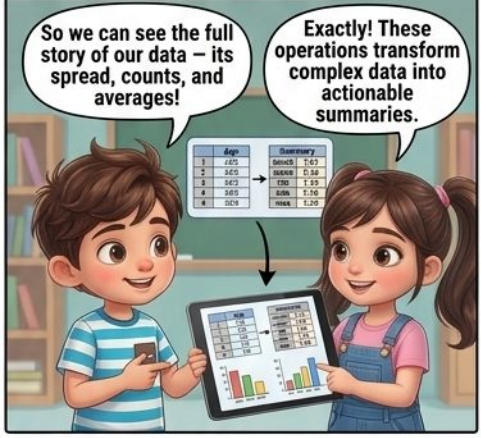
1. Introducing Summarizing & Descriptive Statistics



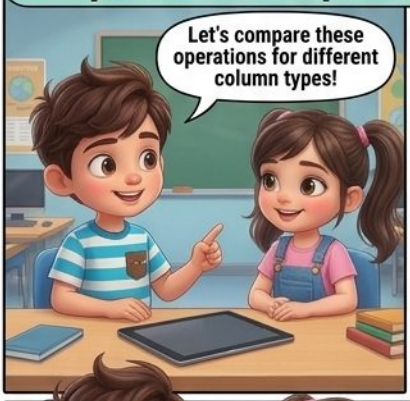


2. Descriptive Methods: Examples & Data Flow





3. Operation Comparison & Takeaway Summary



Pandas Statistical Operation Comparison

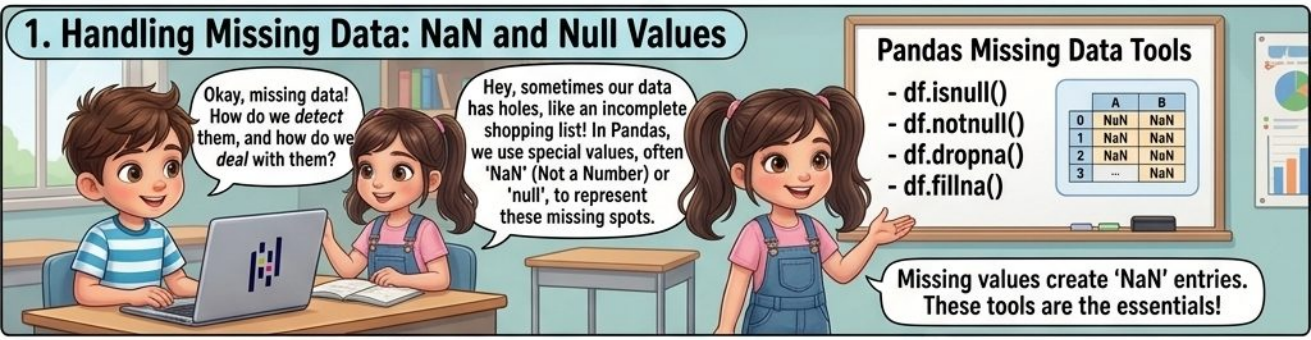
Operation Name	Applies To	Main Output	Pandas Function
Value Counts	Categorical Column	Frequency of categories	df['col'].value_counts()
Describe	Numerical Column	Full Overview (mean, std, min, max, etc.)	df['col'].describe()
Mean	Numerical Column	Arithmetic Average	df['col'].mean()
Sum	Numerical Column	Total Sum	df['col'].sum()

Descriptive operations are applied *per column* to get unique insights. Use the right tool for the data type!



Handling Missing Data in Pandas

1. Handling Missing Data: NaN and Null Values



2. Missing Data Methods: Detection & Cleaning



Methods for handling

(A) `df.isnull().sum()`

	A	B
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN

→ Series

1	True
2	?
3	NaN

`df.isnull()` creates a mask. `.sum()` counts missing values per column.

(B) `df.dropna()`

	A	B
1	NaN	...
2	...	...

→

	A	C
1	...	...

`df.dropna()` removes rows with *any* missing data.

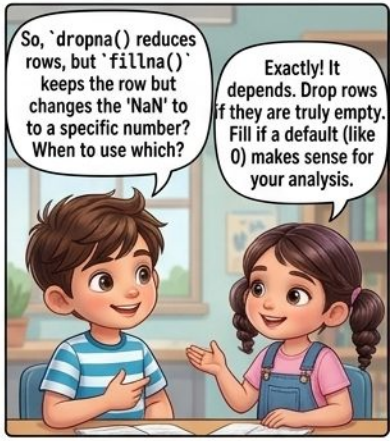
(C) `df.fillna(0)`

	A	B
1	...	NaN
2	...	...

→

	A	C
1	...	0
2	...	0

`df.fillna(value)` replaces missing spots with a specific value.



3. Operation Comparison & Takeaway Summary



Comparison of Missing Data Handling

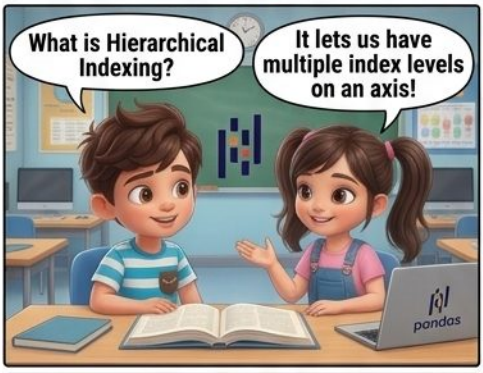
Operation Name	Applies To	Main Output	Pandas Function
isnull() / notnull()	All values	True/False matrix	`df.isnull()`
dropna()	Removal	True/False matrix	`df.dropna()`
fillna()	Imputation	Faluse matrix	`df.fillna(0`

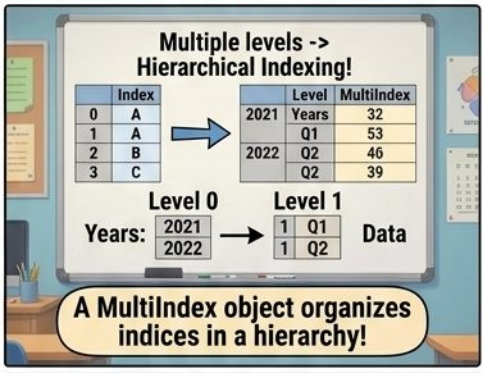
Handle your 'holes'! Choose the right tool for your specific cleaning challenge. Clean data, clear insights!



Hierarchical Indexing in Pandas

1. Introducing Hierarchical Indexing (MultiIndex)





2. Creating & Reshaping with MultiIndex

Methods for creation & reshaping

(A) pd.MultiIndex.from_tuples()

```
[('2021', 'A'), ('2021', 'B')]
[('2021', 'A'), ('2021', 'B')]
[('2021', 'A'), ('2021', 'B')]
```

→ Creates MultiIndex directly from tuple pairs.

(B) df.stack()

	Sales	Sales	Profit
1	2021	5.00	2.60
2	2021	10.00	1.50
3	2021	12.00	5.30
4	2021	2500	8.30

→

		Values
Metric		300
Metric	Q1	1.60
	Q1	1.20
	Q2	4.50
	Q2	5.20

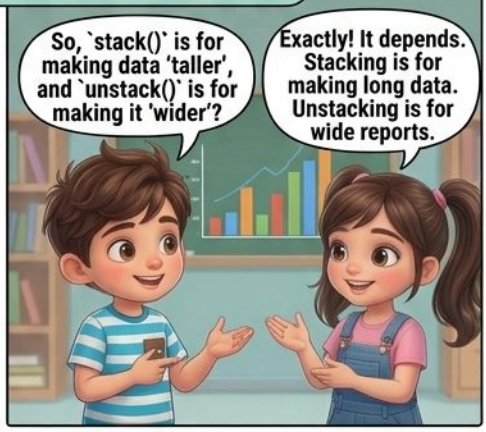
(C) df.unstack()

		Sales	Profit
Years 0		6.00	2.80
Test	Q1	9.00	5.30
	Q2	8.60	3.30
	Q2	1300	31.00

→

Metric	Sales	Profit	Values

`unstack()` moves an index level to column labels.



3. Single vs MultiIndex Comparison & Takeaway



Single vs MultiIndex Operations

Operation Name	Single Index	MultiIndex	Related Pandas Function
Selection	loc['key']	loc['level0', 'level1']	df.loc[]
Aggregation	sum	loc['level0, 'level1']	df.sum[]
Reshaping	stack	loc['level0', 'level1']	df.stack]

Master MultiIndex in Pandas for sophisticated data organization and powerful reshaping!

